

ECE 385

Fall 2024

Final Project

Audio Effects Processor

Ashmit Dutta and Samuel Lee
TA Email: xuanboj2@illinois.edu

1. Introduction

In the ECE 385 final project, we have implemented a fully functional FPGA-based digital audio playback and effects processing system. At the core of the design, audio samples are stored in external DDR3 memory which is then streamed through a pipeline of digital signal processing modules that produce modified PWM signals that are linked through the audio output pin to be heard. This design implements the basic functionalities of a DJ controller that can loop, speed-up/slow-down, and filter incoming music. By integrating an SD card interface, a memory controller, and user controllers through switches and hex display, we are able to implement a full framework for audio playback and manipulation.

2. Written Description and Diagrams

a. Summary of Operation

Upon power-up and reset, the system initializes the DDR3 memory controller (MIG) and the SD card. Once the SD card is ready, a state machine automatically reads raw binary audio data from the card and writes it into DDR3 memory. This loads a large audio dataset without manual intervention. As a result, we were able to play 3 minutes of Passionfruit by Drake completely on our FPGA.

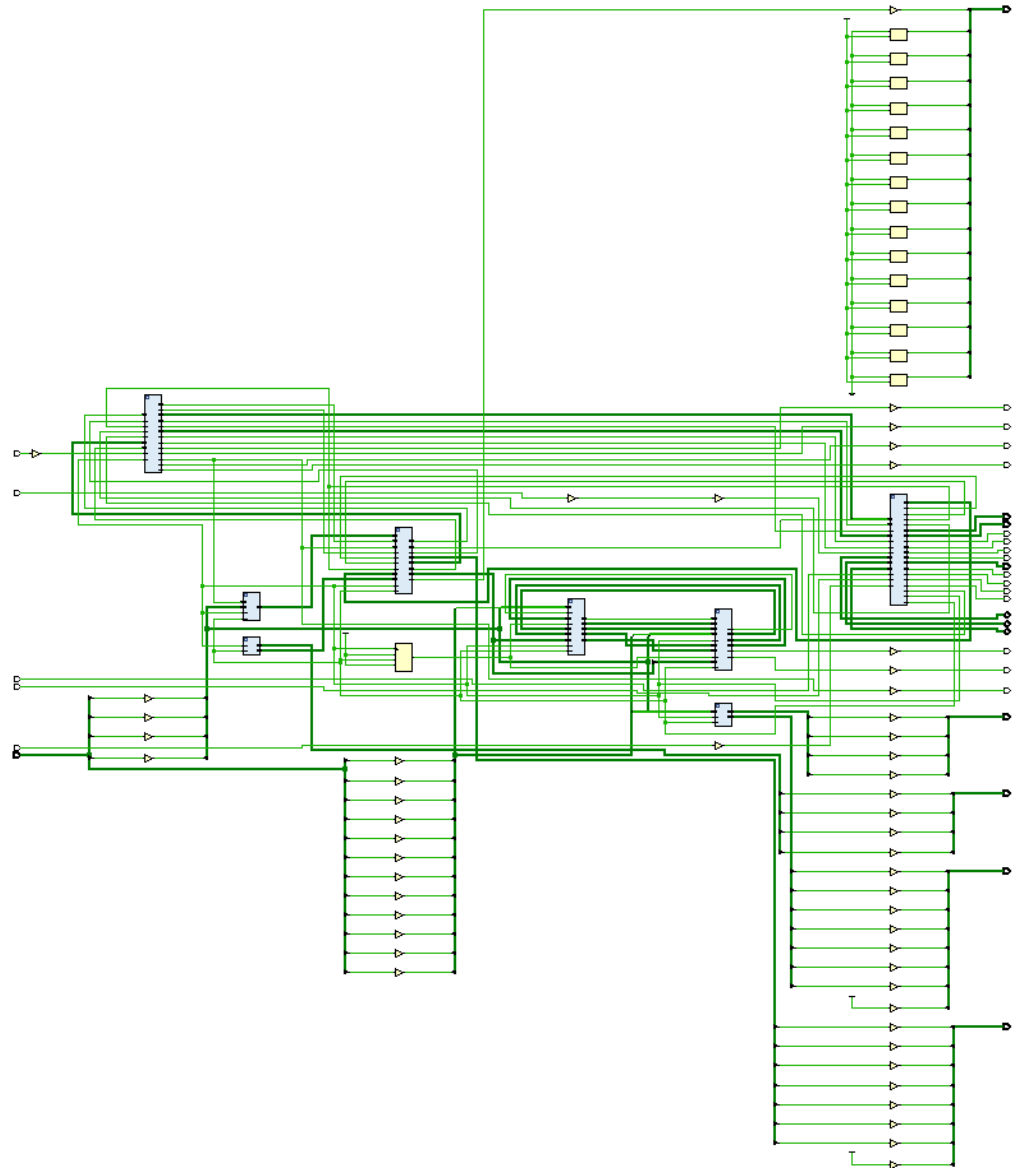
An address incrementer module continuously generates the read addresses for retrieving audio samples from DDR3. Users can adjust playback speed, set loop start/end points, and break out of loops via on-board switches. The system responds by modifying the generated read addresses to loop a specific audio segment or return to normal sequential playback.

Before audio samples reach the output, an optional FIR filter and possibly other effects, like echo, can be applied. These processing steps modify the frequency response or add time-based effects, improving audio quality or introducing creative sound design.

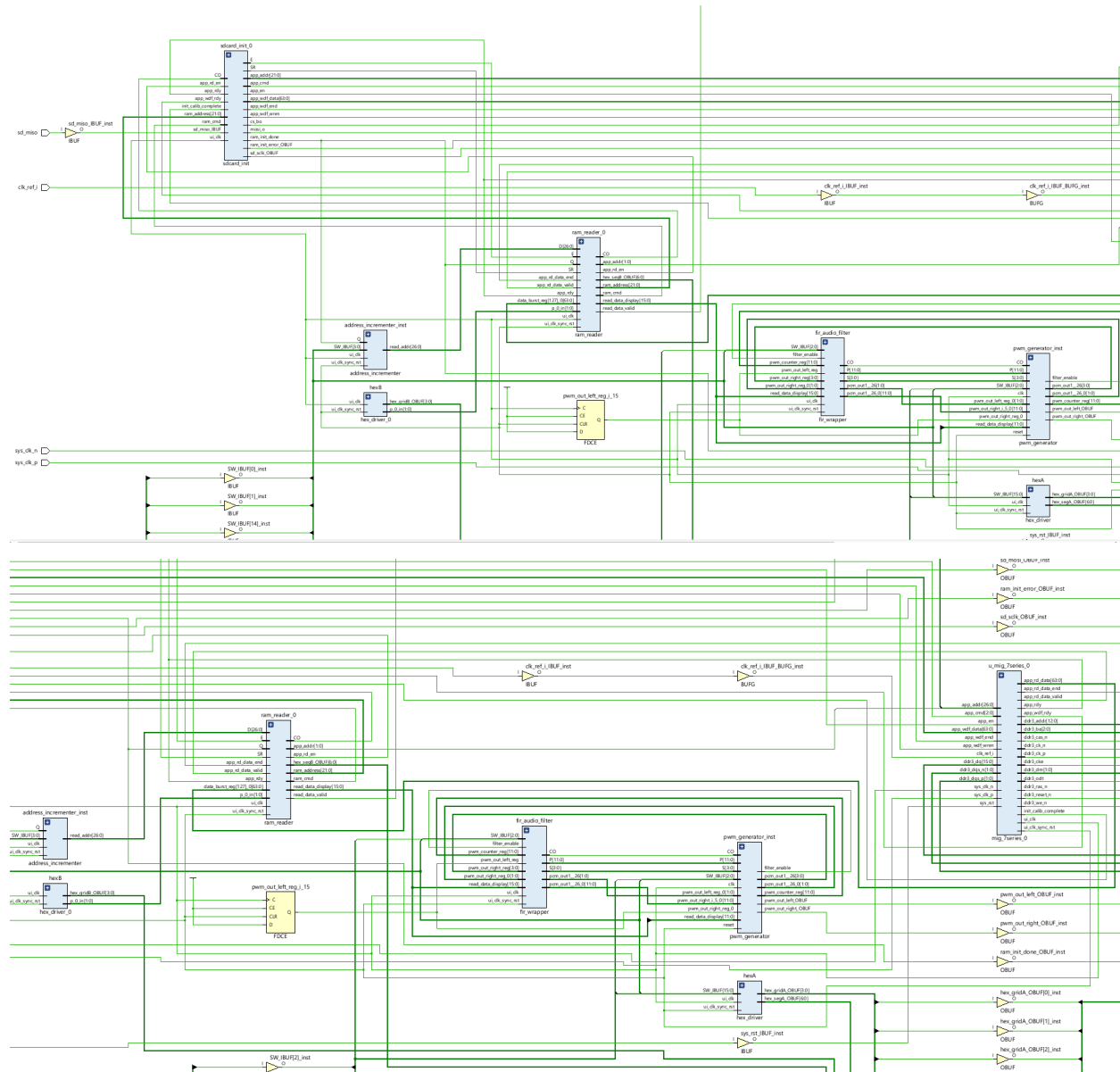
The processed 16-bit PCM samples are converted to a PWM signal suitable for external audio output. Concurrently, the system displays addresses, status indicators, or sample values on HEX displays and LEDs. These visual cues help users monitor the internal state, check for errors, or confirm looping and speed adjustments.

b. Block Diagram View(top-level and individual module if have any)

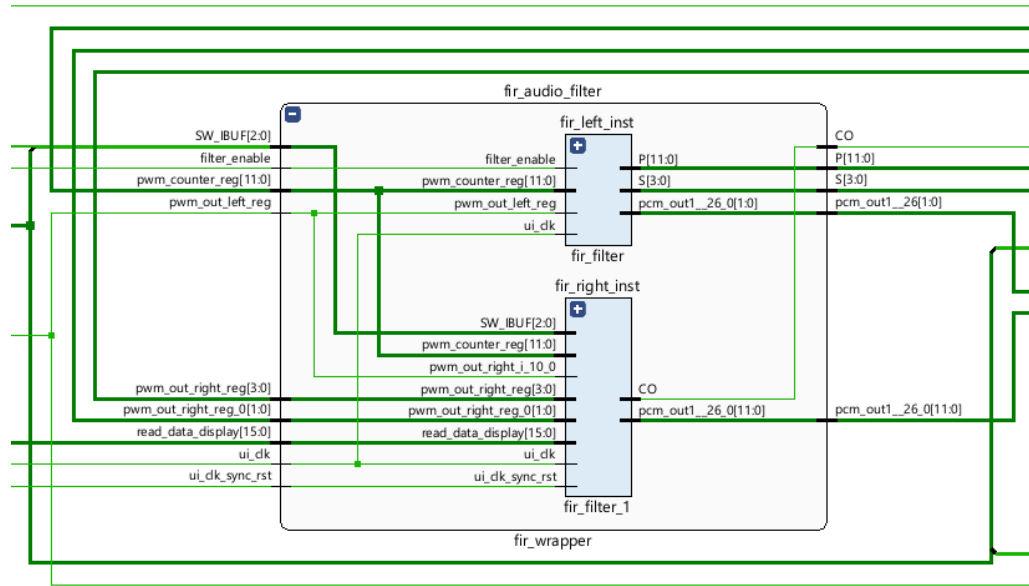
Block design (Top=level)



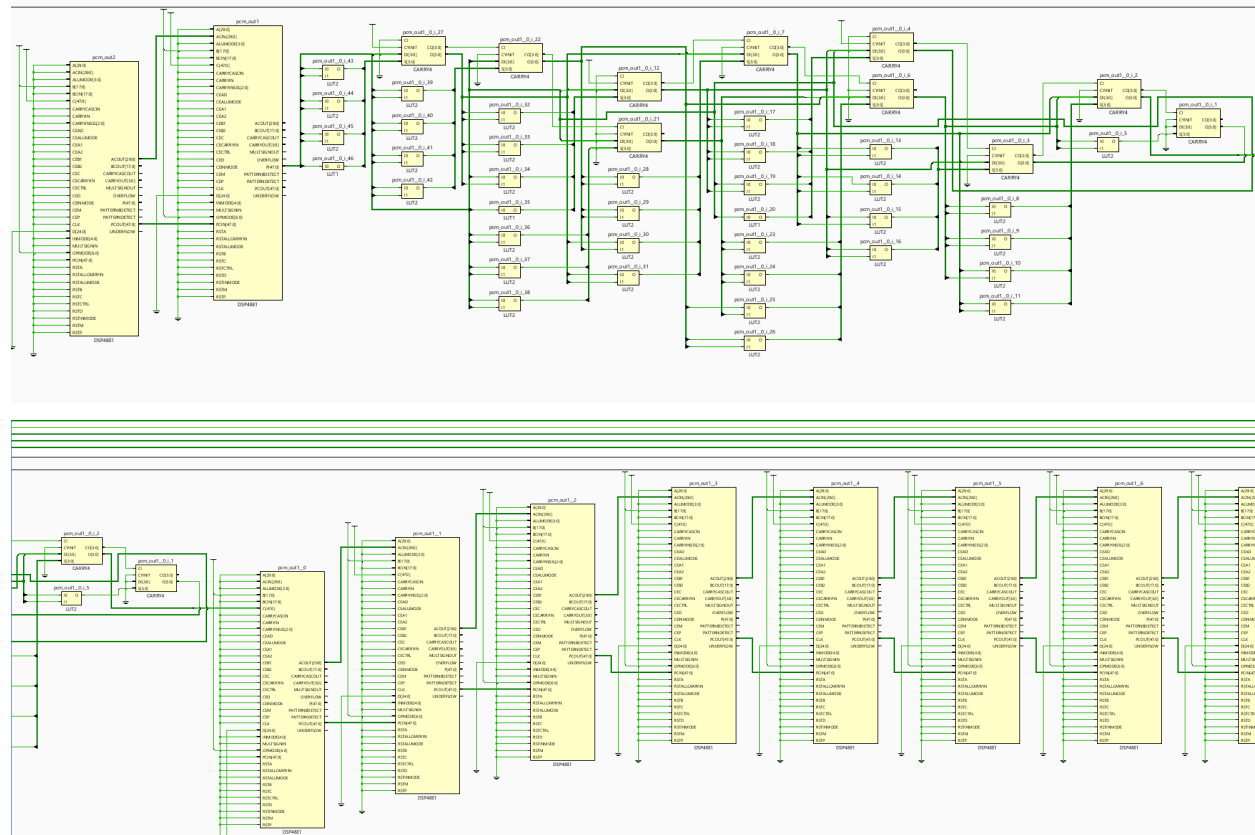
Zoomed in:

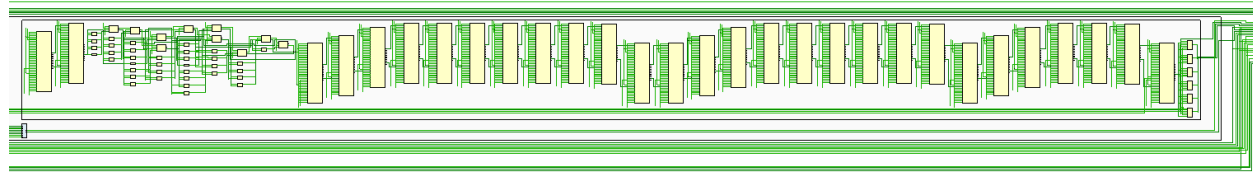


FIR Filter module

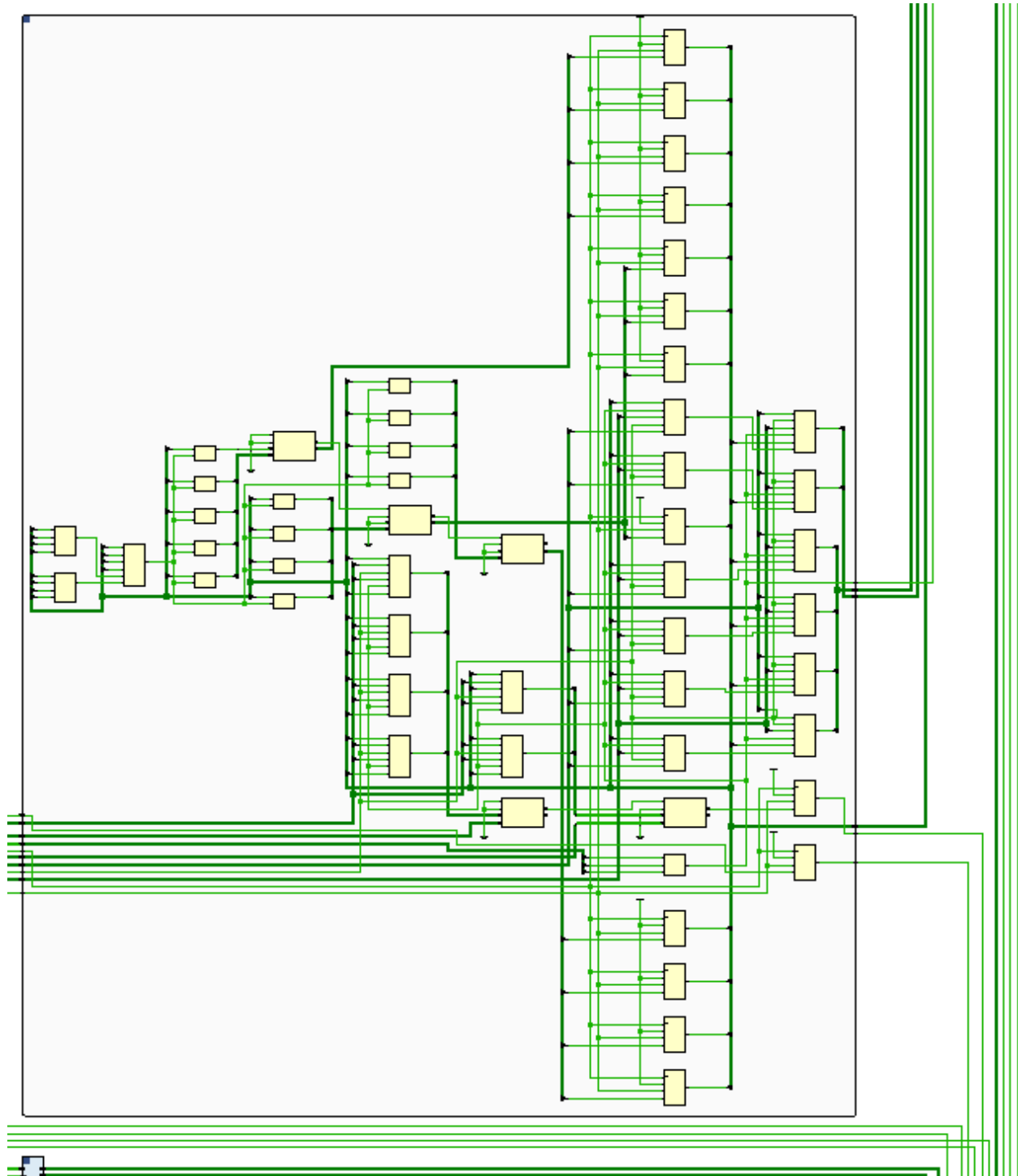


Zoomed in version of fir_left_inst:

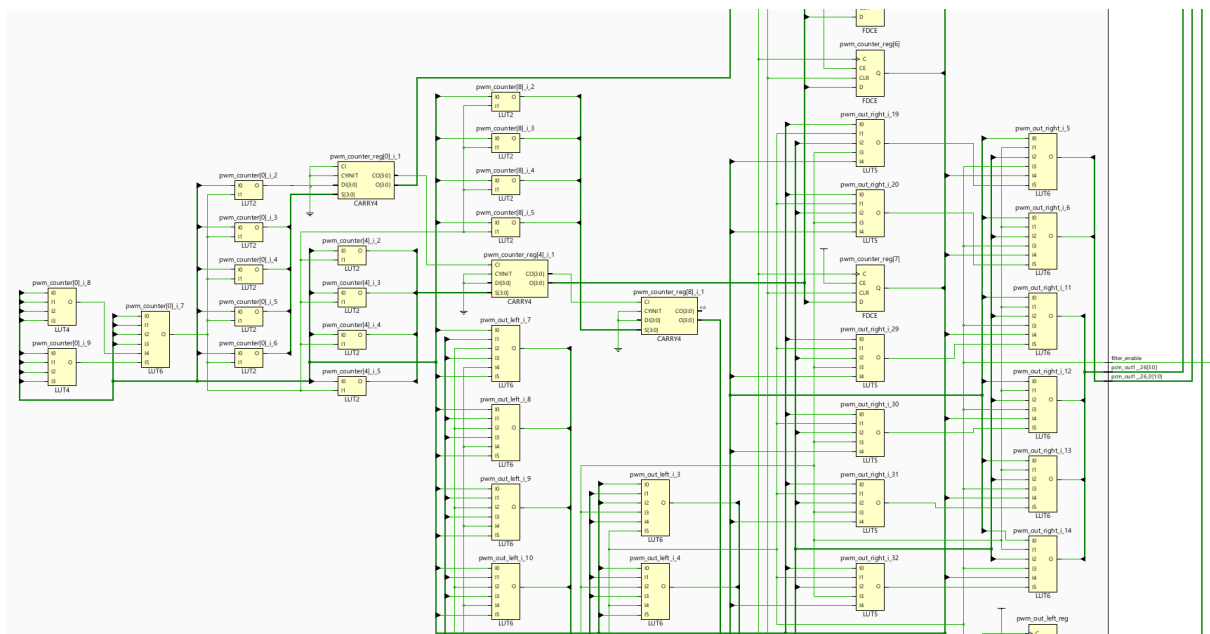




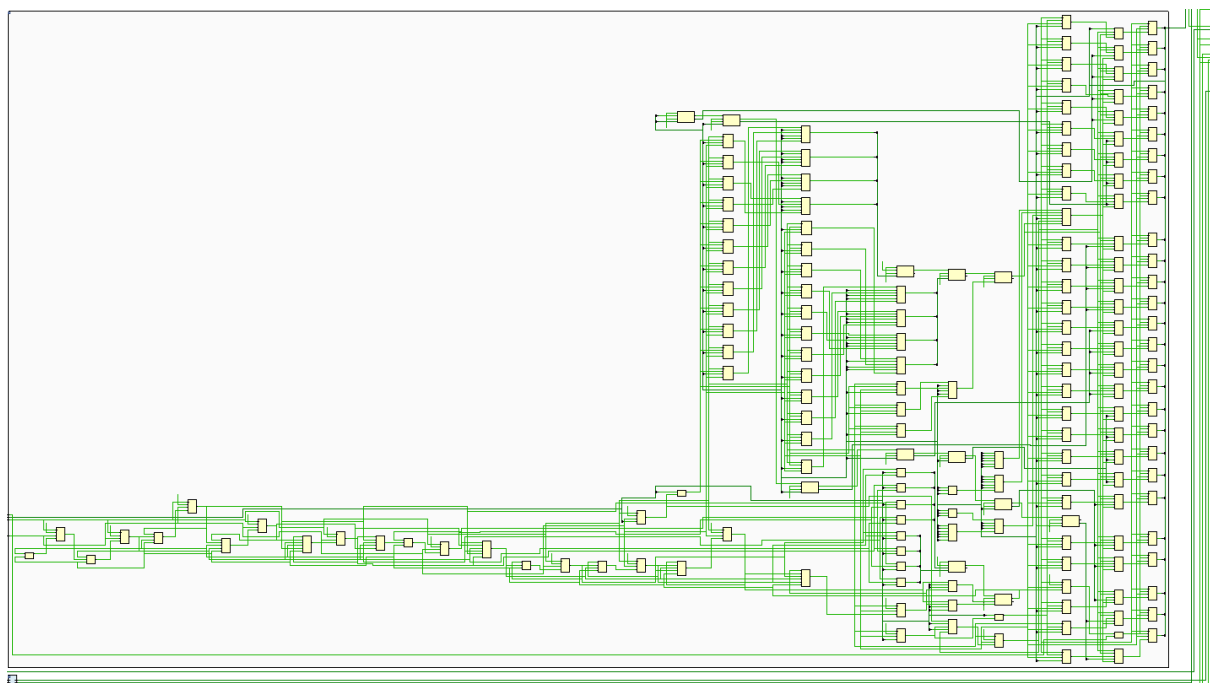
PWM generator module:



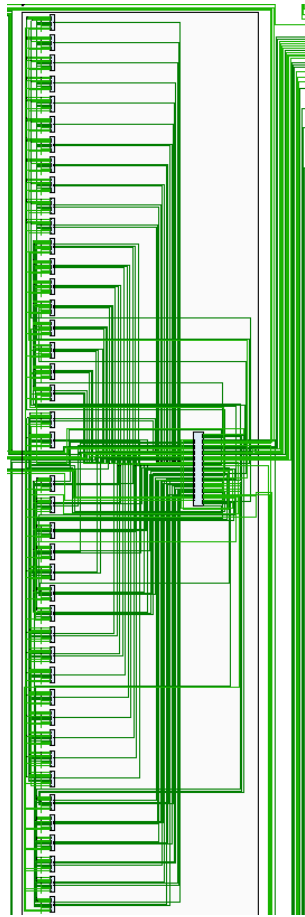
Zoomed in version:



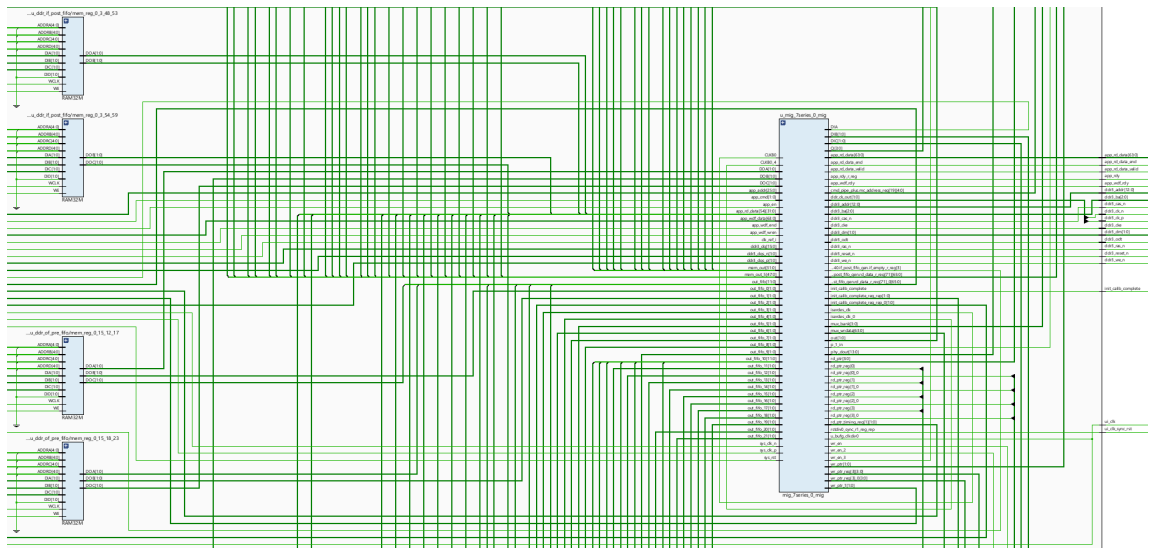
Address_incrementer module



Mig_7series (IP)



Zoomed in version:



c. Description for all .sv module

i. rtl_ddr3_top.sv

- Inputs: sys_clk_n, sys_clk_p, clk_ref_i, sys_rst, sd_miso, [15:0] SW

- Outputs: [12:0] ddr3_addr, [2:0] ddr3_ba, ddr3_cas_n, ddr3_ck_n, ddr3_dk_p, ddr3_cke, [1:0] ddr3_dm, ddr3_odt, ddr3_ras_n, ddr3_reset_n, ddr3_we_n, sd_sclk, sd_mosi, sd_cs, [7:0] hex_segA, [7:0] hex_segB, [3:0] hex_gridA, [3:0] hex_gridB, ram_init_error, ram_init_done, logic[15:0] LED
- Input wires: [15:0] ddr3_dq, [1:0] ddr3_dqs_n, [1:0] ddr3_dqs_p
- Description: This module integrates DDR3 memory control and SD card interfacing to emulate a ROM-like behavior. It initializes the SDHC card, loads data into DDR3, and allows for RAM read operations. It also drives hexadecimal displays and LEDs for debugging or data visualization.
- Purpose: To create a top-level system for interfacing with DDR3 memory, SD card, hex display, and LEDs in FPGA designs.

ii. **sdcard_init.sv**

- Inputs: clk, reset, ram_rdy, ram_wdf_rdy, miso_i
- Outputs: [26:0] ram_address, [2:0] ram_cmd, ram_en, [63:0] ram_wdf_data, ram_wdf_wren, ram_wdf_end, ram_init_error, ram_init_done, cs_bo, sclk_o, mosi_o
- Description: This module identifies if the card inserted is SD or SDHC card and initializes the card appropriately. It reads the card contents directly into DDR3 memory. It uses a state machine to manage SD card operations, handle data transfers, and write data into RAM in bursts of 128 bits.
- Purpose: To facilitate seamless transfer of binary data from an SDHC card to DDR3 memory, enabling embedded systems to use external data sources effectively.

iii. **ram_reader.sv**

- Inputs: clk, reset, ram_rdy, ram_rd_valid, ram_rd_data_end, [63:0] ram_rd_data, [26:0] read_address
- Outputs: [26:0] ram_address, [2:0] ram_cmd, ram_en, [15:0] read_data_out, read_data_valid
- Description: This module interfaces with DDR3 memory to read 16-bit wave data. It handles burst reads and manages address updates. A 128-bit burst of data is fetched, and 16-bit words are output sequentially.
- Purpose: To facilitate efficient and customizable memory read operations, particularly useful in applications where continuous or burst data retrieval is required.

iv. **hex_driver.sv**

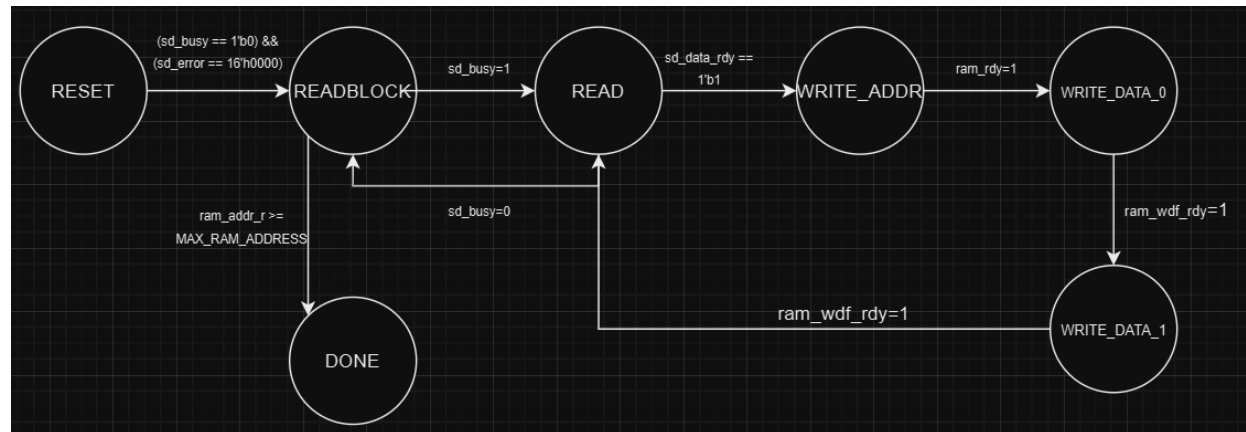
- Inputs: clk, reset, [3:0] in [3:0]
- Outputs: [7:0] hex_seg, [3:0] hex_grid
- Description: This module is responsible for driving two 7-segment displays based on the value of the input val. It converts the binary input into a corresponding set of segments to light up on the displays.

- Purpose: The main purpose of this module is to interface with two 7-segment displays, converting binary input values into a readable format for display purpose on the FPGA. It helps visualize data during operations like multiplication or addition.
- v. **address_incrementer.sv**
 - Inputs: clk, reset, enable, [1:0] speed_select, sw_set_loop_start, sw_set_loop_end, sw_break_loop
 - Outputs: [ADDR_WIDTH-1:0] read_addr, read_addr_valid
 - Description: Generates a continuously incrementing address to read audio samples from memory. It can do a loop feature between two addresses set via switches, allowing the address to wrap back to a start point once it reaches an end point. Additionally, it can adjust the increment rate based on the speed_select input, enabling dynamic playback speeds.
 - Purpose: To be able to stream audio samples and stream the pwm module which is essential for audio playback and also basic effects.
- vi. **fir_filter.sv**
 - Inputs: clock, reset, [WIDTH-1:0] pcm_in
 - Outputs: [WIDTH-1:0] pcm_out
 - Description: Applies a convolution of predefined coefficients on the audio source so different frequencies can get filtered out.
 - Purpose: Apply a low pass filter on the incoming audio.
- vii. **fir_wrapper.sv**
 - Inputs: clk, reset, filter_enable, [15:0] pcm_in_left, [15:0] pcm_in_right
 - Outputs: [15:0] pcm_out_left, [15:0] pcm_out_right
 - Description: Applies a FIR filter on each audio channel. Either pass the incoming samples directly if filter_enable is low or run them through the FIR filter.
 - Purpose: Provides simple/enable interfacing for the FIR filters.
- viii. **pwm_generator.sv**
 - Inputs: clk, reset, [15:0] pcm_left, [15:0] pcm_right
 - Outputs: pwm_out_left, pwm_out_right
 - Description: Converts PCM signals into PWM signals. These are interfaced with the B13 and B14 audio out pins.
 - Purpose: To be able to hear audio through the FPGA.
- ix. **mig_7series_0.sv (IP)**
 - Inputs: sys_clk_p, sys_clk_n, clk_ref_i, sys_rst, [26:0] app_addr, [2:0] app_cmd, app_en, [63:0] app_wdf_data, app_wdf_end, [7:0] app_wdf_mask, app_wdf_wren, app_sr_req, app_ref_req, app_zq_req, [15:0] ddr3_dq, [1:0] ddr3_dqs_n, [1:0] ddr3_dqs_p

- Outputs: [12:0] ddr3_addr, [2:0] ddr3_ba, ddr3_ras_n, ddr3_cas_n, ddr3_we_n, ddr3_reset_n, [0:0] ddr3_ck_p, [0:0] ddr3_ck_n, [0:0] ddr3_cke, [1:0] ddr3_dm, [0:0] ddr3_odt, [63:0] app_rd_data, app_rd_data_end, app_rd_data_valid, app_rdy, app_wdf_rdy, app_sr_active, app_ref_ack, app_zq_ack, ui_clk, ui_clk_sync_rst, init_calib_complete, [11:0] device_temp
- Description: The MIG handles all aspects of DDR3 control, including initialization, refresh cycles, read/write scheduling, and calibration routines.
- Purpose: To abstract the complexities of DDR3 SDRAM interaction, allowing the rest of the system to read and write memory using straightforward commands.

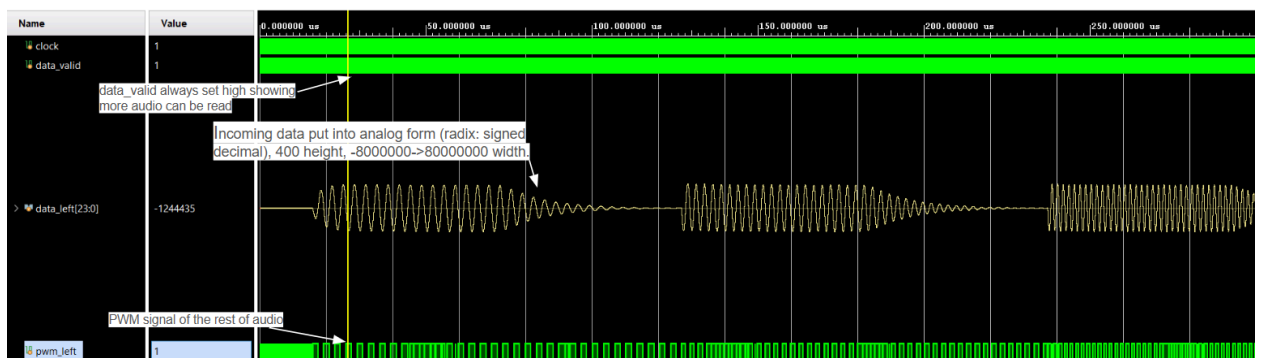
d. State Diagram

sdcard_init.sv: The system begins by initializing the SD card and preparing the DDR3 memory for data storage. Upon reset, the logic checks if the SD card is accessible and ready. If the card is successfully initialized, the system enters a state machine that continuously reads data blocks from the SD card and writes them into DDR3 memory. Each block read from the card is transferred into memory in 128-bit bursts, ensuring efficient high-speed data throughput.



3. Simulation with annotation

Here is an example of how we can parse through the WAV file to get incoming data.

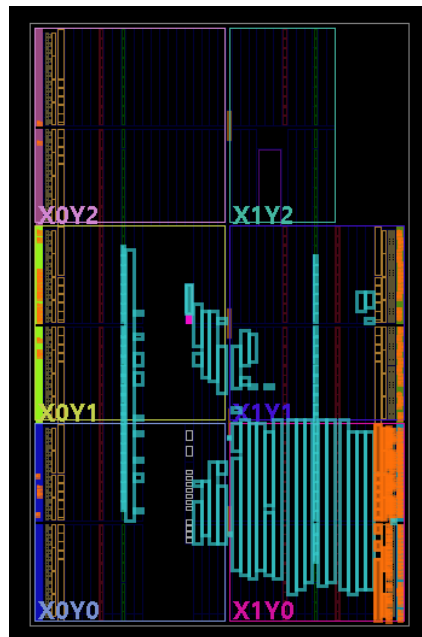


We cannot simulate the rest of the functions because it has to be interfaced with an SD card and can only be put on an FPGA to test if it works.

4. Design Resource and Statistics

LUT	3721
DSP	58
Memory	268 LUT used as memory
Flip-Flop	3868
Latches*	0
Frequency (1/(T-WNS))	1.56 GHz
Static Power	0.078 W
Dynamic Power	0.895 W
Total Power	0.973 W

Implemented Design



5. Conclusion

We successfully built an FPGA-based audio effects processor that reads audio from an SD card, stores it in DDR3 memory, and plays it back with looping and filtering features. Through this project, we learned how to manage DDR3 memory and handle audio signals using PCM and PWM formats. Our project allows users to control playback speed and loop points with onboard

switches, and outputs audio through PWM signals which allows for integration of memory management, audio processing, and user controls within an FPGA.